
SkillFlow: Long-Horizon Agent Execution with Residual Context

Anonymous Author(s)

Affiliation

Address

email

Abstract

1

2 1 Method

3 We propose **SkillFlow**, a general framework for long-horizon agent execution that improves per-
4 formance by organizing agent behavior around structured skill transitions and a *residual context*
5 *pathway*. The key idea is to avoid relying on ever-growing raw interaction history as the sole decision
6 substrate. Instead, SkillFlow decomposes complex tasks into skill-level execution units, maintains
7 a persistent representation of user-critical intent, and compresses long interaction trajectories into
8 compact, goal-aware, and risk-aware execution states.

9 The framework is motivated by a fundamental failure mode of long-horizon agents: as context grows,
10 user goals, constraints, and verified task state become diluted among noisy intermediate reasoning
11 traces, stale observations, and irrelevant tool outputs. This often leads to goal drift, hallucinated
12 continuation, and unreliable actions. Inspired by residual learning, SkillFlow introduces a *residual*
13 *context memory* that preserves high-value task information across long execution horizons, ensuring
14 that critical user intent remains directly accessible at every step.

15 1.1 Problem Formulation

16 We consider a long-horizon agent that interacts with a user and an external environment over T steps.
17 Let x denote the initial user request, and let e_t denote the environment observation at step t , which
18 may include tool outputs, retrieved documents, file-system responses, or other task-relevant signals.
19 The agent produces an action a_t at each step.

20 In conventional long-context agents, the next action is generated from the full accumulated interaction
21 history,

$$a_t \sim \pi_\theta(\cdot \mid H_t), \quad (1)$$

22 where

$$H_t = \{x, (a_1, e_1), (a_2, e_2), \dots, (a_{t-1}, e_{t-1})\}. \quad (2)$$

23 As t increases, H_t becomes increasingly long and heterogeneous, mixing user intent, tool outputs,
24 stale intermediate states, and model-generated reasoning. This creates three coupled failure modes:

$$\text{Long-Horizon Failure} \Leftarrow \text{Goal Dilution} + \text{State Inconsistency} + \text{Hallucinated Continuation}. \quad (3)$$

25 Our goal is to construct an execution policy that remains faithful to the user’s objective and robust
26 to context growth. Instead of conditioning solely on raw history, SkillFlow maintains a structured
27 execution state:

$$a_t \sim \pi_\theta(\cdot \mid G, S_t, R_t, E_t), \quad (4)$$

28 where:

- 29 • G is the persistent **goal anchor** extracted from the user’s request,
- 30 • S_t is the current **skill state**,
- 31 • R_t is the **residual context memory** that preserves user-critical information and compressed
- 32 execution state,
- 33 • E_t is the local environment context relevant to the current step.

34 The core challenge is to design R_t such that it is compact enough to prevent context explosion, while
 35 remaining faithful to the information necessary for long-horizon execution.

36 1.2 Overview of SkillFlow

37 SkillFlow consists of five components:

- 38 1. **Goal Anchoring**: parses the user request into a persistent structured goal representation.
- 39 2. **Skill Planning**: decomposes complex tasks into a sequence of structured skills.
- 40 3. **Local Skill Execution**: executes the current skill using local context and retrieved memory,
- 41 rather than the full raw history.
- 42 4. **Residual Context Memory**: maintains a compact residual pathway that preserves user-
- 43 critical intent and execution-critical state.
- 44 5. **Risk-Aware Action Gating**: ensures that high-impact actions are consistent with goal
- 45 constraints and current execution state.

46 At a high level, SkillFlow replaces token-level history accumulation with *skill-level state transition*.
 47 Rather than continually appending interaction traces into a single prompt, the framework updates
 48 a structured memory state after each skill, and only retrieves the information most relevant to the
 49 current execution decision.

50 1.3 Goal Anchoring

51 Given the initial user request x and optional later user clarifications, SkillFlow constructs a persistent
 52 goal anchor

$$G = \text{Anchor}(x), \quad (5)$$

53 where

$$G = (G^{\text{obj}}, G^{\text{hard}}, G^{\text{pref}}, G^{\text{risk}}). \quad (6)$$

54 Here:

- 55 • G^{obj} denotes the core task objective,
- 56 • G^{hard} denotes hard constraints that must never be violated,
- 57 • G^{pref} denotes soft user preferences,
- 58 • G^{risk} denotes action boundaries and confirmation requirements for potentially high-impact
- 59 operations.

60 This decomposition is critical for long-horizon execution. In particular, we distinguish between:

$$G^{\text{hard}} \cap G^{\text{pref}} = \emptyset, \quad (7)$$

61 since hard constraints require lossless preservation, while soft preferences may be abstracted or
 62 prioritized during compression.

63 Unlike conventional memory summaries that may paraphrase or weaken user requirements, Goal
 64 Anchoring explicitly preserves irreversible constraints such as:

- 65 • *do not delete permanently,*
- 66 • *ask before sending messages,*
- 67 • *do not make purchases without confirmation,*
- 68 • *do not disclose private information.*

69 1.4 Skill Planning and Execution

70 SkillFlow models long-horizon tasks as structured skill transitions. A skill is defined as

$$s_k = (n_k, g_k, p_k, u_k, z_k, c_k), \quad (8)$$

71 where:

- 72 • n_k is the skill name,
- 73 • g_k is the local subgoal,
- 74 • p_k is the precondition,
- 75 • u_k is the execution action or tool invocation,
- 76 • z_k is the expected output schema,
- 77 • c_k is the completion condition.

78 Given the global goal G and current residual memory R_t , the planner selects the next skill:

$$S_t = \text{PlanSkill}(G, R_t, E_t). \quad (9)$$

79 The key design choice is that a skill is executed only with *local context*:

$$C_t^{\text{local}} = [G, S_t, \text{Retrieve}(R_t, S_t), E_t]. \quad (10)$$

80 This prevents the model from repeatedly reprocessing the entire raw interaction history, which often
81 contains substantial irrelevant or stale information.

82 After skill execution, the environment returns an observation e_t , and the framework updates its
83 residual memory state:

$$e_t = \text{Execute}(S_t, C_t^{\text{local}}). \quad (11)$$

84 1.5 Residual Context Memory

85 The central mechanism of SkillFlow is a **residual context memory** that preserves user-critical
86 information across long interaction horizons. Inspired by residual learning, the goal is to ensure that
87 important signals remain directly accessible even after many skill transitions.

88 Instead of relying on

$$H_t = H_{t-1} \cup \{(a_{t-1}, e_{t-1})\}, \quad (12)$$

89 SkillFlow maintains a structured residual state

$$R_t = (R_t^{\text{goal}}, R_t^{\text{exec}}, R_t^{\text{risk}}), \quad (13)$$

90 with three channels:

- 91 • **Goal Residual** R_t^{goal} : persistent user objective, hard constraints, and core preferences,
- 92 • **Execution Residual** R_t^{exec} : compressed skill-level task state,
- 93 • **Risk Residual** R_t^{risk} : unresolved high-impact conditions, missing confirmations, and blocked
94 actions.

95 The next residual state is updated as

$$R_t = \text{UpdateResidual}(R_{t-1}, S_t, e_t, G). \quad (14)$$

96 This residual pathway plays a role analogous to information-preserving skip connections: it allows
97 the agent to continue planning over long horizons without losing access to high-value user and task
98 signals.

99 **Goal Residual.** The goal residual is initialized from the goal anchor:

$$R_0^{\text{goal}} = \text{CompressGoal}(G). \quad (15)$$

100 It is designed to be compact but high-fidelity, preserving especially the set of hard constraints:

$$R_t^{\text{goal}} \supseteq G^{\text{hard}}, \quad \forall t. \quad (16)$$

101 Thus, any compression step must preserve all hard constraints exactly.

102 **Execution Residual.** The execution residual stores the minimal state necessary for reliable continu-
 103 ation:

$$R_t^{\text{exec}} = \{m_1, m_2, \dots, m_t\}, \quad (17)$$

104 where each memory item m_i is a compressed skill state:

$$m_i = (n_i, g_i, o_i^{\text{key}}, v_i, \ell_i, q_i). \quad (18)$$

105 Here:

- 106 • o_i^{key} is the key outcome,
- 107 • v_i denotes evidence or verification signals,
- 108 • ℓ_i denotes execution status (e.g., success, failed, pending),
- 109 • q_i denotes unresolved issues or next-step dependencies.

110 **Risk Residual.** The risk residual explicitly stores high-impact unresolved conditions:

$$R_t^{\text{risk}} = \text{CompressRisk}(S_t, e_t, G). \quad (19)$$

111 Examples include:

- 112 • deletion requested but confirmation missing,
- 113 • payment available but user approval absent,
- 114 • file ownership or identity ambiguous,
- 115 • evidence insufficient for a high-impact action.

116 1.6 Goal-Aware Compression Schema

117 A central design principle in SkillFlow is that compression should preserve *decision-critical state*,
 118 not merely produce a shorter textual summary. We therefore define compression as a goal-aware
 119 structured transformation:

$$R_t = f_{\text{comp}}(H_t, G), \quad (20)$$

120 where f_{comp} must satisfy three properties:

Goal-Preserving: $\text{Info}(G^{\text{hard}}; R_t)$ is maximal, (21)

State-Faithful: R_t accurately reflects the latest validated execution state, (22)

Risk-Aware: R_t explicitly retains unresolved high-impact conditions. (23)

121 We compress different information types differently:

122 **(1) Immutable user-critical information.** Hard constraints, confirmation requirements, sensitive
 123 boundaries, and identity bindings are preserved with near-lossless fidelity:

$$\mathcal{C}_{\text{hard}}(u) = u, \quad \forall u \in G^{\text{hard}}. \quad (24)$$

124 **(2) Structured execution state.** Completed skill outcomes are transformed into schema-aligned
 125 memory entries:

$$\mathcal{C}_{\text{exec}}(S_t, e_t) \rightarrow m_t = (\text{subgoal}, \text{result}, \text{status}, \text{evidence}, \text{pending}). \quad (25)$$

126 **(3) Dropable information.** The following are aggressively compressed or removed:

- 127 • verbose chain-of-thought,
- 128 • repeated observations,
- 129 • stale intermediate hypotheses,
- 130 • details superseded by validated later state.

131 This yields a memory representation that is significantly more compact than raw interaction history
 132 while remaining more faithful to long-horizon execution needs than generic text summarization.

1.7 Retrieval from Residual Memory

At step t , the agent does not consume the entire residual memory. Instead, it retrieves the subset relevant to the current skill:

$$\hat{R}_t = \text{Retrieve}(R_t, S_t), \quad (26)$$

where $\hat{R}_t$ typically contains:

- the persistent goal residual,
- the most recent skill summaries relevant to S_t ,
- unresolved risk entries associated with the current action.

The resulting execution prompt is therefore:

$$C_t^{\text{exec}} = [R_t^{\text{goal}}, \hat{R}_t^{\text{exec}}, \hat{R}_t^{\text{risk}}, S_t, E_t]. \quad (27)$$

This design reduces prompt entropy while keeping the information most relevant for correct continuation.

1.8 Risk-Aware Action Gating

Although SkillFlow is not solely a safety method, reliable long-horizon execution requires preventing unsupported high-impact actions. We therefore introduce a lightweight action gate:

$$a_t = \Pi_{\text{gate}}(\tilde{a}_t, R_t, G), \quad (28)$$

where $\tilde{a}_t$ is a candidate action proposed by the planner or executor, and Π_{gate} checks consistency with:

- hard constraints in G ,
- unresolved risk entries in R_t^{risk} ,
- missing evidence or confirmation conditions.

If a candidate action violates execution requirements, the gate transforms the action into a safer alternative such as:

- request user confirmation,
- retrieve additional evidence,
- defer or abstain from execution.

This mechanism improves execution reliability while remaining a secondary consequence of the broader goal-faithful execution framework.

1.9 Optimization Objective

SkillFlow may be instantiated either as an inference-time framework or as a learned compression-and-execution policy. In the learned setting, we optimize for task success, goal fidelity, execution reliability, and compactness:

$$\mathcal{L} = \lambda_1 \mathcal{L}_{\text{task}} + \lambda_2 \mathcal{L}_{\text{goal}} + \lambda_3 \mathcal{L}_{\text{risk}} + \lambda_4 \mathcal{L}_{\text{state}} + \lambda_5 \mathcal{L}_{\text{len}}. \quad (29)$$

Here:

- $\mathcal{L}_{\text{task}}$ measures task completion quality,
- $\mathcal{L}_{\text{goal}}$ penalizes violation or forgetting of user-critical intent,
- $\mathcal{L}_{\text{risk}}$ penalizes unsupported high-impact actions,
- $\mathcal{L}_{\text{state}}$ penalizes inconsistency between compressed memory and validated execution state,
- $\mathcal{L}_{\text{len}}$ penalizes excessive memory length.

This objective formalizes the compression tradeoff: memory must be compact, but not at the cost of losing goal fidelity or execution-critical state.

Algorithm 1 SkillFlow for Long-Horizon Agent Execution

Require: User request x , environment $\mathcal{E}$, execution horizon T

```
1:  $G \leftarrow \text{Anchor}(x)$ 
2:  $R_0^{\text{goal}} \leftarrow \text{CompressGoal}(G)$ 
3:  $R_0^{\text{exec}} \leftarrow \emptyset$ 
4:  $R_0^{\text{risk}} \leftarrow \emptyset$ 
5: for  $t = 1, 2, \dots, T$  do
6:    $E_t \leftarrow \text{Observe}(\mathcal{E})$ 
7:    $\hat{R}_t \leftarrow \text{Retrieve}(R_{t-1}, E_t)$ 
8:    $S_t \leftarrow \text{PlanSkill}(G, \hat{R}_t, E_t)$ 
9:    $C_t^{\text{local}} \leftarrow [R_{t-1}^{\text{goal}}, \hat{R}_t^{\text{exec}}, \hat{R}_t^{\text{risk}}, S_t, E_t]$ 
10:   $\tilde{a}_t, e_t \leftarrow \text{Execute}(S_t, C_t^{\text{local}})$ 
11:   $m_t^{\text{exec}} \leftarrow \text{CompressExec}(S_t, e_t)$ 
12:   $m_t^{\text{risk}} \leftarrow \text{CompressRisk}(S_t, e_t, G)$ 
13:   $R_t^{\text{goal}} \leftarrow \text{UpdateGoalResidual}(R_{t-1}^{\text{goal}}, G)$ 
14:   $R_t^{\text{exec}} \leftarrow \text{UpdateExecResidual}(R_{t-1}^{\text{exec}}, m_t^{\text{exec}})$ 
15:   $R_t^{\text{risk}} \leftarrow \text{UpdateRiskResidual}(R_{t-1}^{\text{risk}}, m_t^{\text{risk}})$ 
16:   $a_t \leftarrow \Pi_{\text{gate}}(\tilde{a}_t, R_t, G)$ 
17:   $\text{Act}(a_t)$ 
18:  if  $\text{Done}(G, R_t)$  then
19:    break
20:  end if
21: end for
22: return Final action trace and residual memory  $R_t$ 
```

1.10 Algorithm

Algorithm 1 summarizes the SkillFlow procedure.

1.11 Discussion

SkillFlow departs from conventional long-context agents in two key ways. First, it treats long-horizon execution as a sequence of structured skill transitions rather than unconstrained continuation over raw text history. Second, it maintains a residual context pathway that preserves user-critical information throughout execution. This design reduces context growth, improves goal retention, and lowers the probability of hallucinated or unsupported actions. Importantly, the resulting gains in execution reliability emerge not from a standalone safety filter, but from a more faithful and better-structured execution substrate for long-horizon agents.

References

A Technical Appendices and Supplementary Material

Technical appendices with additional results, figures, graphs and proofs may be submitted with the paper submission before the full submission deadline (see above), or as a separate PDF in the ZIP file below before the supplementary material deadline. There is no page limit for the technical appendices.